

High-Performance Data Analysis

(Análise de Dados de Elevado Desempenho)

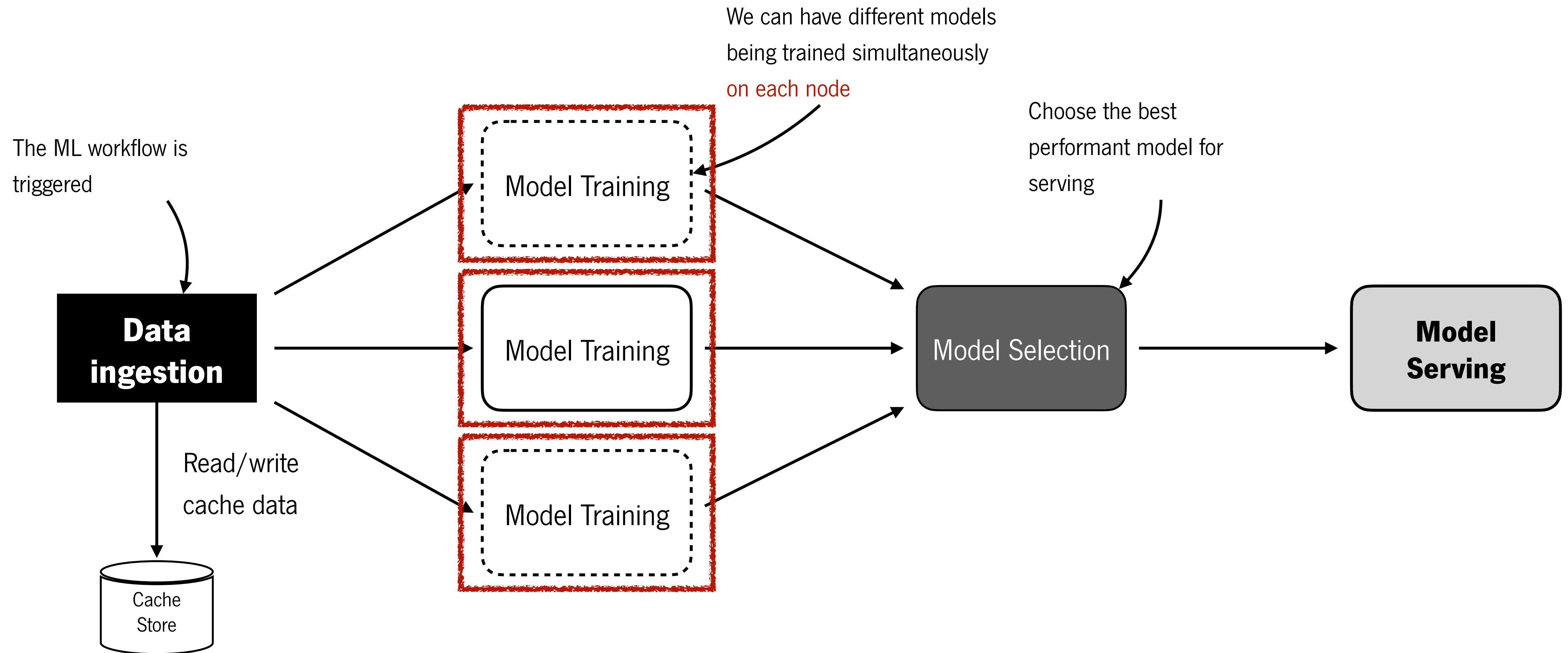
Distributed Training

University of Minho

2025-2026



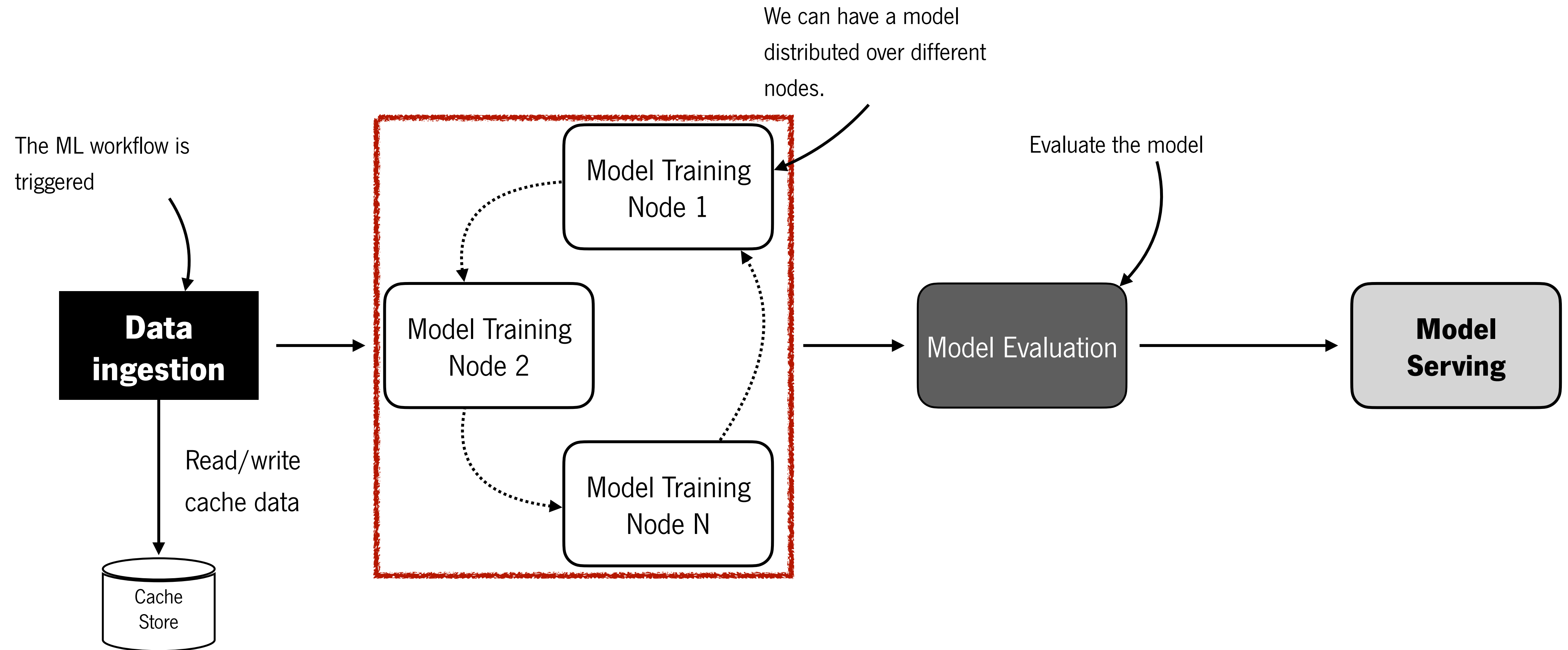
From Single-Node to Distributed Training



Data ingestion is the first step in a machine learning pipeline.

The rest of the processes in the machine learning pipeline are all dependent on the success of data ingestion.

From Single-Node to Distributed Training



Data ingestion is the first step in a machine learning pipeline.

The rest of the processes in the machine learning pipeline are all dependent on the success of data ingestion.

Why Distributed Training?

◎ **Large datasets and models:**

- OpenImages (561GB).
- LAION-5B (5 billion images - 340TB).
- GPT3 (175B parameters).
- DeepSeek - R1 (671B parameters— **not even the biggest!**).

◎ **Faster convergence** (distributed training reduces wall time significantly).

A Taxonomy of Parallelism in ML

● Data Parallelism

- When we have large amounts of data and small models.

● Model Parallelism

- Very large models with billions of parameters.

● Pipeline Parallelism

- Large models with sequential layers.

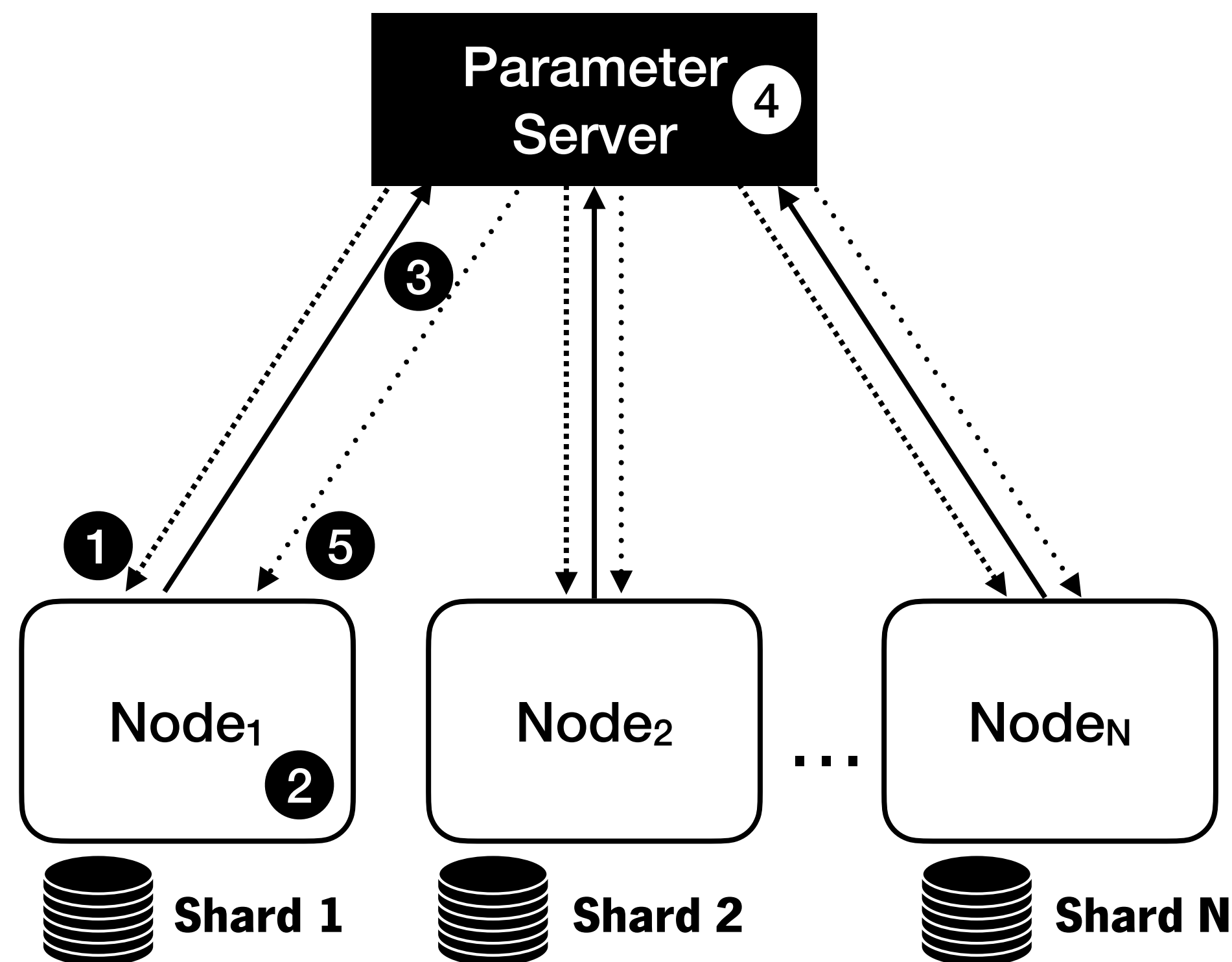
● Hybrid Parallelism

- The combination of data and model parallelism

A Taxonomy of Parallelism in ML

● Data Parallelism

- ▶ When we have large amounts of data and small models.



- 1 The parameter sends the model to each node.
- 2 Train model on its shard of data.
- 3 Send to parameter server the trained parameters.
- 4 Aggregate and/or average trained parameters.
- 5 Send new trained parameters to the nodes.
- 6 The cycle repeats until the train is finished and the parameter server as a final trained model.

A Taxonomy of Parallelism in ML

◎ Data Parallelism

➡ Synchronous and Asynchronous

- **Synchronous Training:** All nodes wait to synchronize the gradients (e.g., AllReduce), increasing stall time.
- **Asynchronous Training:** Nodes train independently, leading to less consistent results.

➡ Gradient Averaging:

- **AllReduce:** Decentralized and fast.
- **Parameter Server:** Centralized and flexible, but can bottleneck.

A Taxonomy of Parallelism in ML

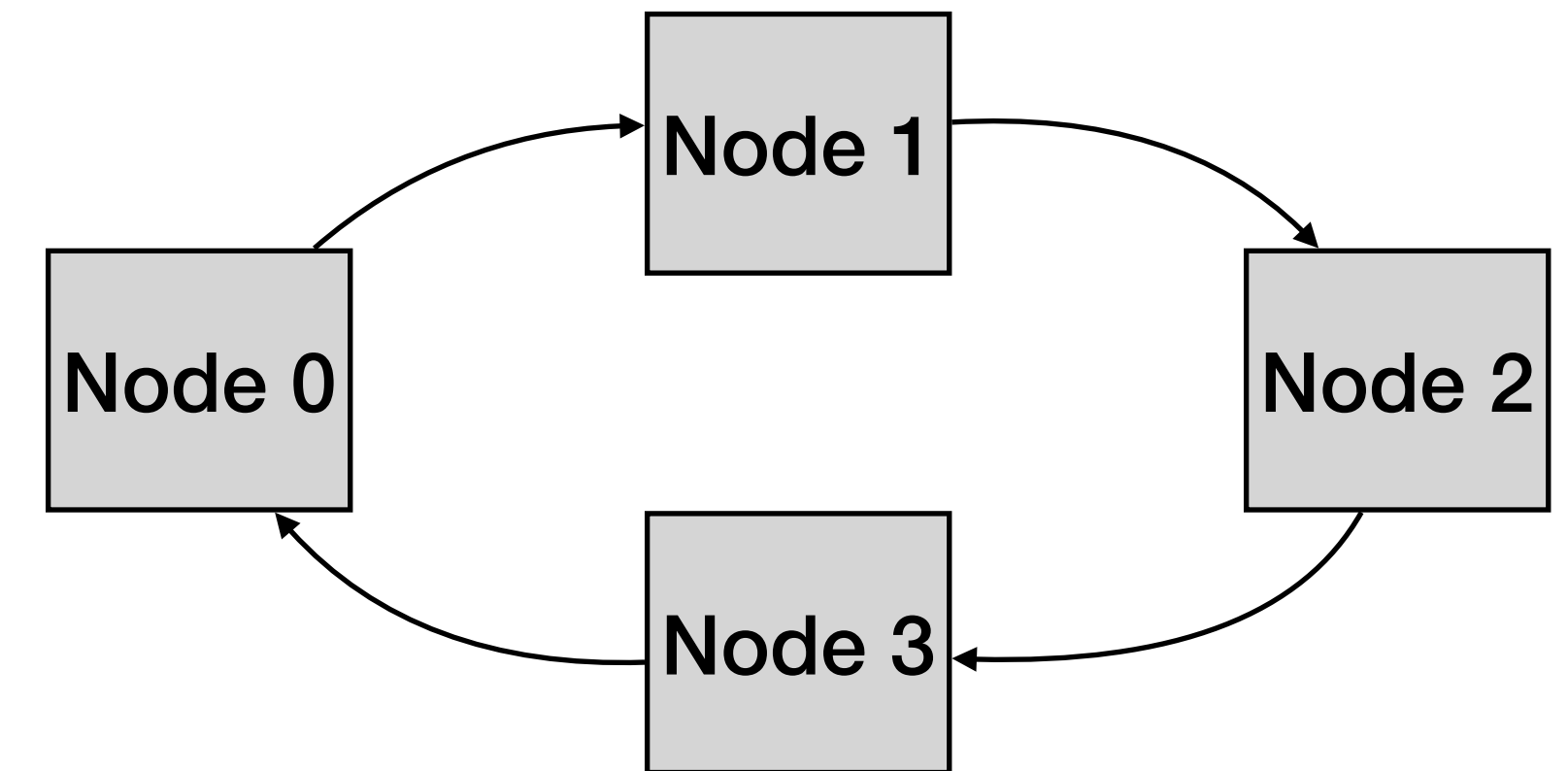
Data Parallelism - AllReduce

- Each node computes **local gradients** from its data shard.
- AllReduce aggregates gradients **across all nodes**.
- The aggregated result is **broadcasted back** to update the replicated models.

Typical operations:

- Sum: Most common in gradient averaging.
- Mean: Divide summed gradients by world size after AllReduce.
- Max/Min/Product: Rare, more common in diagnostics.

Ring AllReduce



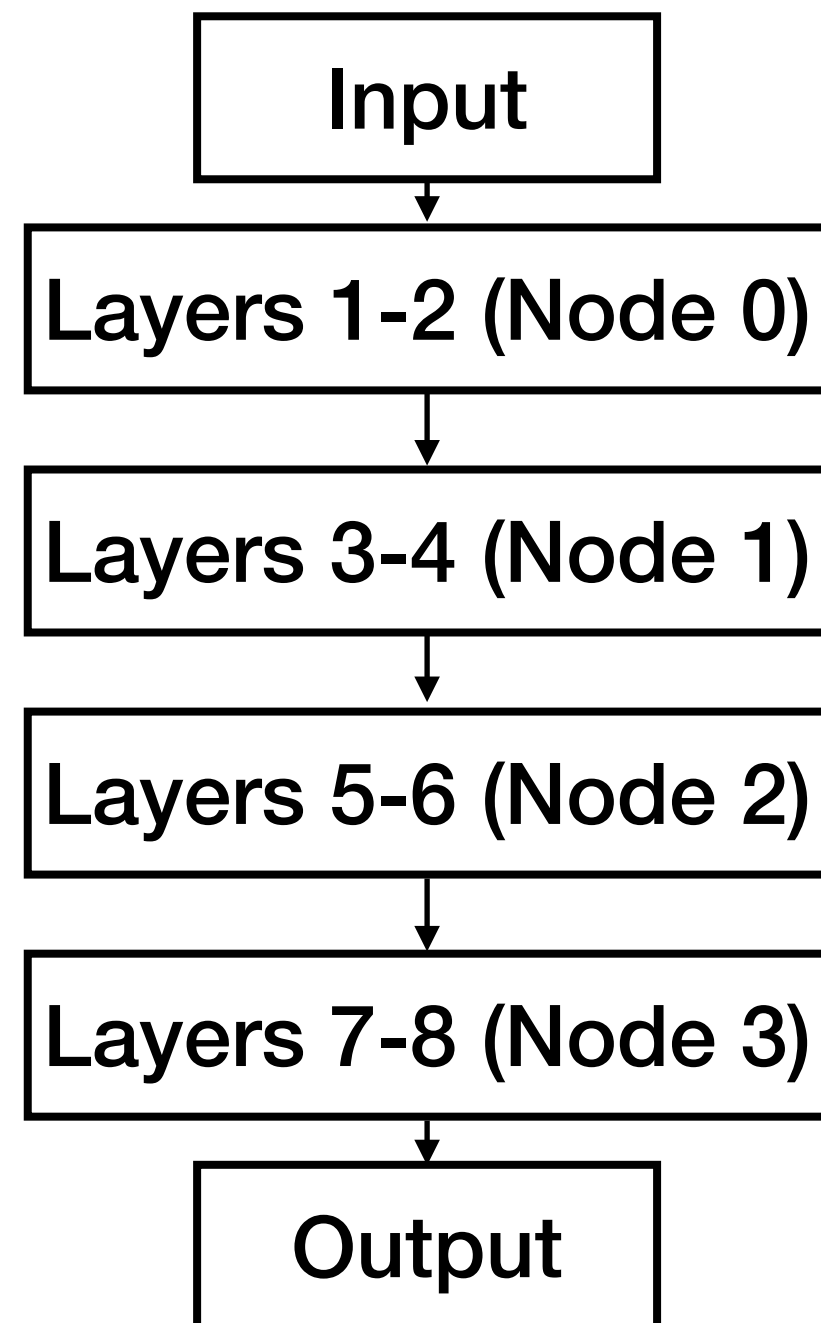
- 1 Each node sends part of its data to the next and receives from the previous.
- 2 Over $N-1$ steps, data is reduced (e.g., added) in a **ring topology**.
- 3 The reduced data is then circulated back to **all nodes**.

A Taxonomy of Parallelism in ML

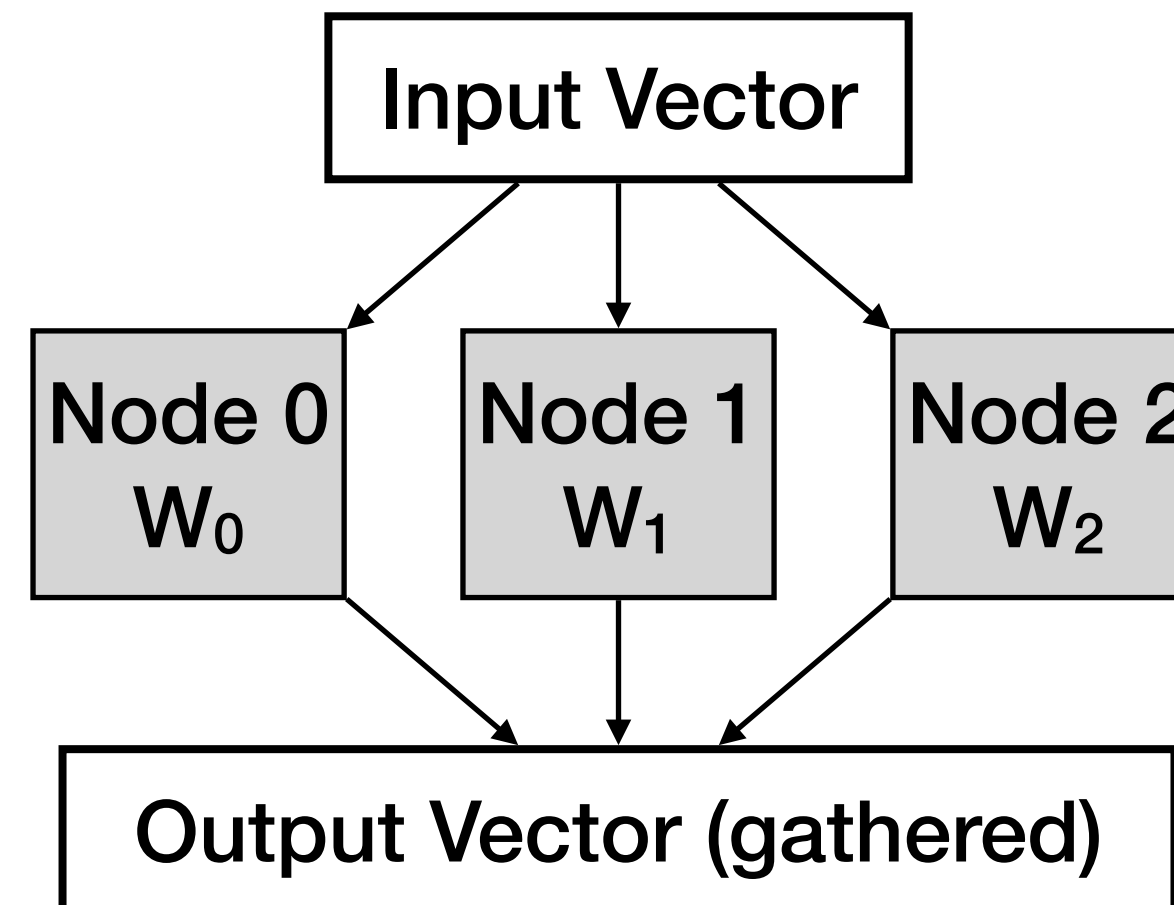
● Model Parallelism

- ▶ Very large models with billions of parameters.

Layer-wise (vertical split)

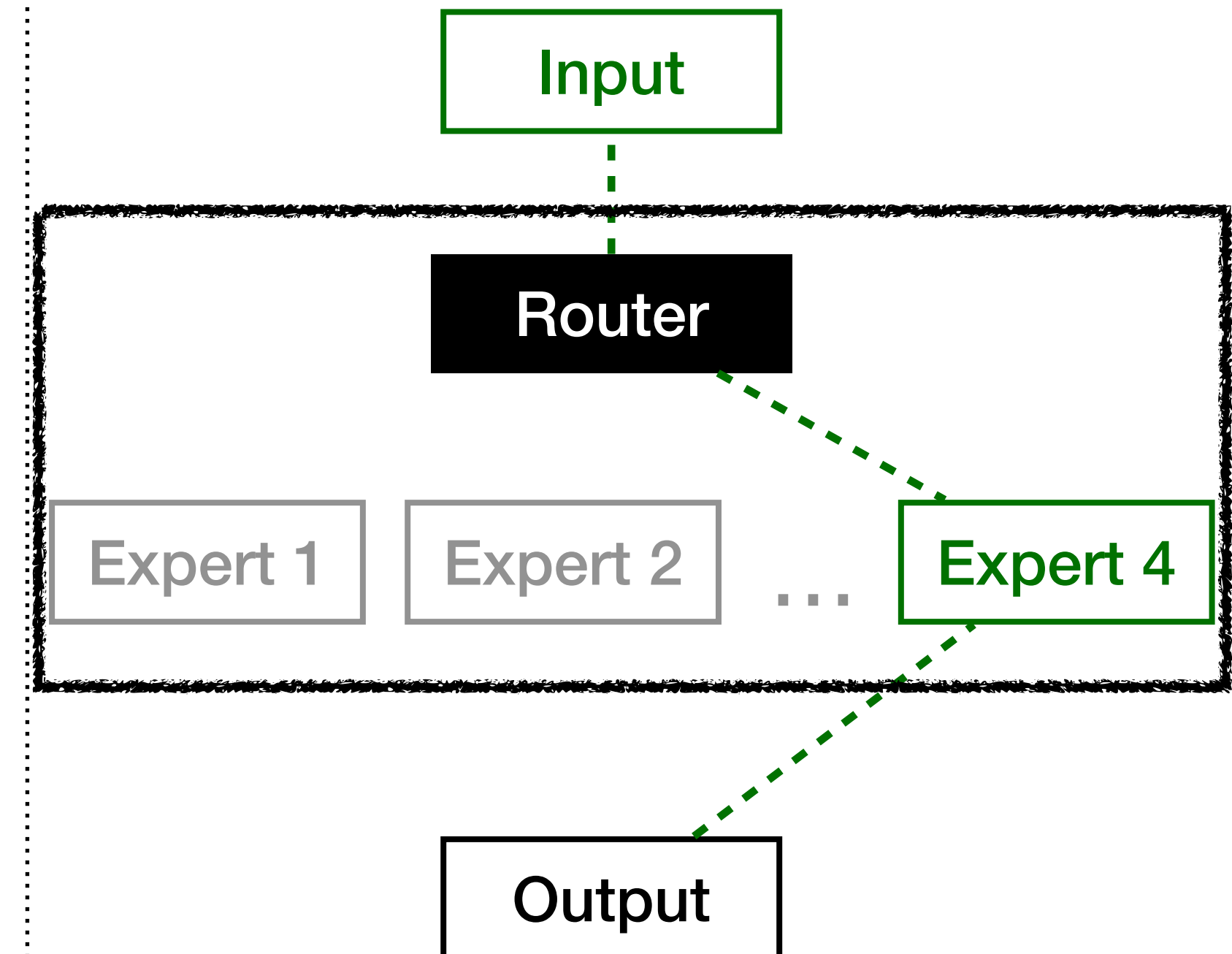


Tensor (horizontal split)



Each Node computes a partial matrix-vector product and contributes to the final output.

Mixture-of-Experts (MoE)



A Taxonomy of Parallelism in ML

◎ Pipeline Parallelism

- ▶ Large models with sequential layers, similar to layer-wise parallelism.
- ▶ The number of micro-batches should be at least $(n+1)$, where n is the number of nodes.

TIME	NODE 0 (STAGE 1)	NODE 1 (STAGE 2)	NODE 2 (STAGE 3)
T1	MB1		
T2	MB2	MB1	
T3	MB3	MB2	MB1
T4	MB4	MB3	MB2
T5		MB4	MB3
T6			MB4

More micro-batches **reduce bubble size** and increase Node utilization!

$$\text{Pipeline Efficiency} = (\text{num_microbatches}) / (\text{num_microbatches} + \text{num_stages} - 1)$$

A Taxonomy of Parallelism in ML

ASPECT	Model Parallelism (Vertical Split)	Pipeline Parallelism
WHAT IS SPLIT	Layers of the model	Layers of the model
EXECUTION SCHEDULING	Each micro-batch flows entirely through the Nodes one after the other	Micro-batches are scheduled in a pipelined and overlapping fashion
PARALLELISM	Only one micro-batch is processed at a time across all Nodes	Multiple micro-batches flow through different Nodes simultaneously
COMPUTE OVERLAP	No overlap, nodes may sit idle until it's their turn	Nodes can all work in parallel on different parts of different micro-batches
LATENCY VS THROUGHPUT	Low latency, poor throughput	Slightly higher latency, much better throughput
EXAMPLE FRAMEWORK	Manual nn.Sequential across devices	PyTorch Pipe, DeepSpeed pipeline engine

PyTorch DistributedDataParallel (DDP)

- Built-in support for multi-GPU/multi-node training.
- Uses NCCL backend for fast Node-to-Node communication.
- Gradients are synced using AllReduce.
- Preferred over **DataParallel** due to better performance and scaling.
 - DataParallel resorts to the parameter server while DDP is fully decentralised by using AllReduce.

DDP Code Example

```
import torch.distributed as dist

from torch.nn.parallel import DistributedDataParallel as DDP


def setup(rank, world_size):
    dist.init_process_group("nccl", rank=rank, world_size=world_size)
    torch.cuda.set_device(rank)

model = MyModel().cuda(rank)

ddp_model = DDP(model, device_ids=[rank])
```

PyTorch DistributedDataParallel (DDP)

- Communication, Sharding, Fault Tolerance
 - ▶ **AllReduce:** Gradients are synchronized across Nodes.
 - ▶ **Sharded DDP:** Reduces memory overhead using FairScale.
 - ▶ **Elastic Training:** PyTorch Elastic enables fault tolerance and scale-up/down.
 - ▶ **Checkpointing:** DDP allows recovery in the event of failures.

Horovod

- Developed by Uber, integrates with PyTorch, TensorFlow, and Keras.
- Uses ring-allreduce instead of parameter server for an efficient gradient communication pattern.
- Supports MPI, NCCL backends.
- Easy integration and elastic training support.

```
import horovod.torch as hvd

hvd.init()

model = Net().cuda()

optimizer = optim.SGD(model.parameters(), lr=0.01 * hvd.size())

optimizer = hvd.DistributedOptimizer(optimizer)

hvd.broadcast_parameters(model.state_dict(), root_rank=0)
```

DeepSpeed: Beyond Parallelism

- Created by Microsoft for massive models like Turing-NLG and BLOOM.
- **ZeRO Optimizer:** Splits optimizer state, gradient, and parameters.
- **ZeRO-Offload:** Moves memory to CPU/NVMe for efficiency.
- **Supports:** automatic mixed precision, MoE, and pipeline parallelism.

```
import deepspeed
from model import MyModel

model = MyModel()
optimizer =
torch.optim.Adam(model.parameters(), lr=3e-4)

# Initialize with DeepSpeed
model_engine, optimizer, _, _ =
deepspeed.initialize(
    args=args,
    model=model,
    optimizer=optimizer,
    model_parameters=model.parameters(),
    config="ds_config.json"
```

Comparison

Feature	DDP	Horovod	DeepSpeed
Ease of use	Medium	Very easy	Medium
Communication	NCCL (AllReduce)	MPI/NCCL (Ring)	NCCL / CPU Offloading
Optimizations	Gradient sync	Ring-allreduce	ZeRO, mixed precision
Scalability	High	High	Very High

Load Balancing Techniques

- **Causes of Imbalance:** Stragglers, data skew, I/O variance.
- **Static vs Dynamic:** Static splits vs adaptive workloads.
- **DeepSpeed's ZeRO-Offload** helps maintain balance with memory optimization.
- **Gradient accumulation** allows large batch training on limited Nodes.

Efficiency Metrics: Speedup, Scaling, Throughput

- ◎ **Speedup** = $\text{Time}(\text{single node}) / \text{Time}(n \text{ nodes})$
- ◎ **Scaling Efficiency** = $\text{Speedup} / \# \text{nodes}$
- ◎ **Throughput** = samples/sec
- ◎ **Weak Scaling:** Keep workload per node constant.
- ◎ **Strong Scaling:** Keep total workload fixed.

Challenges and Bottlenecks

- **Network Bandwidth:** Becomes a bottleneck during gradient synchronization.
- **Stragglers:** The slowest node delays the step.
- **Debugging:** Failures in multi-node settings are complex, even when using checkpoints.
- **Memory Usage:** Optimizer + gradient + activation growth is exponential.

Best Practices

- **Always measure before optimizing.**

- ▶ Profile I/O, compute, communication.
- ▶ Identify bottlenecks in computation and communication.
- ▶ Monitor node utilization and memory consumption.

- Prefer NCCL for intra-node communication (NVLink is expensive but is the best way to make GPUs communicate with each other).

- Use multiple workers per node wisely.

Questions?